Lesson 6 - Scheduling Algorithms

FCFS(First Come First Serve)

- Jobs are executed on first come, first serve basis.
- It is a non-preemptive algorithm.
- Easy to understand and implement.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

1. **Arrival:** Processes enter the system and are placed in a queue in the order they arrive.
2. **Execution:** The CPU takes the *first process* from the front of the queue, *executes it until it is complete*, and then removes it from the queue.
3. **Repeat:** The CPU takes the next process in the queue and repeats the execution process.

Steps in FCFS

1. Find execution order of processes

- The execution order depends on the arrival time.
- The processes are ordered from lowest arrival time to largest.
- Should arrival times be similar, the processes are ordered according to PID.

  Ex:

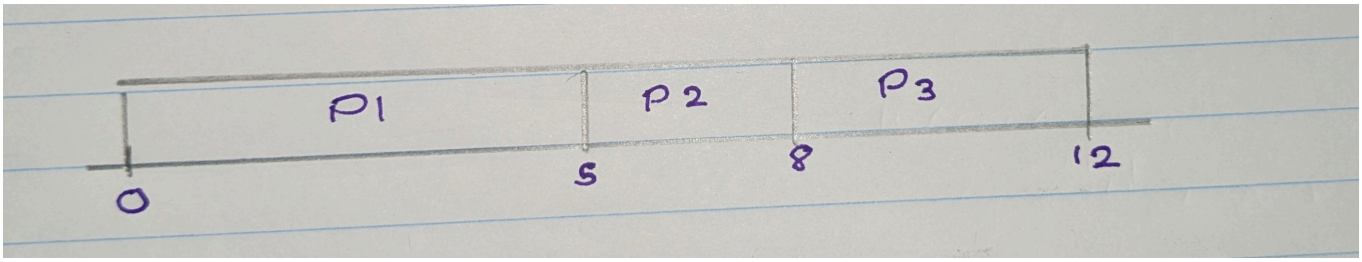
Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	2 ms
P2	3 ms	0 ms
P3	4 ms	4 ms

- The execution order is P2 -> P1 -> P3

Process	Arrival Time	Burst Time
p1	0	5
p2	0	3
p3	0	8

- The execution order is P1 -> P2 -> P3

2. Draw the Gantt chart with the timeline



3. Calculate Completion times

- The completion times for each process are the points in the timeline where the process completed execution.
- Ex: In the above Gantt chart, P0 ended on 5, hence the completion time is 5, while P2 ended on 8, making its completion time 12.

4. Calculate Turnaround times

> Turnaround Time = Completion Time - Arrival Time

- Ex:
- Turnaround times for the above chart are,
  P1 = 5-0 = 5
  P2 = 8-1 = 7
  P3 = 12-2 = 10

5. Calculate Waiting times

> Waiting Time = Turnaround Time - Burst Time

- Ex:
  P1 = 5-5 = 0
  P2 = 7-3= 4
  P3 = 10-4 = 6

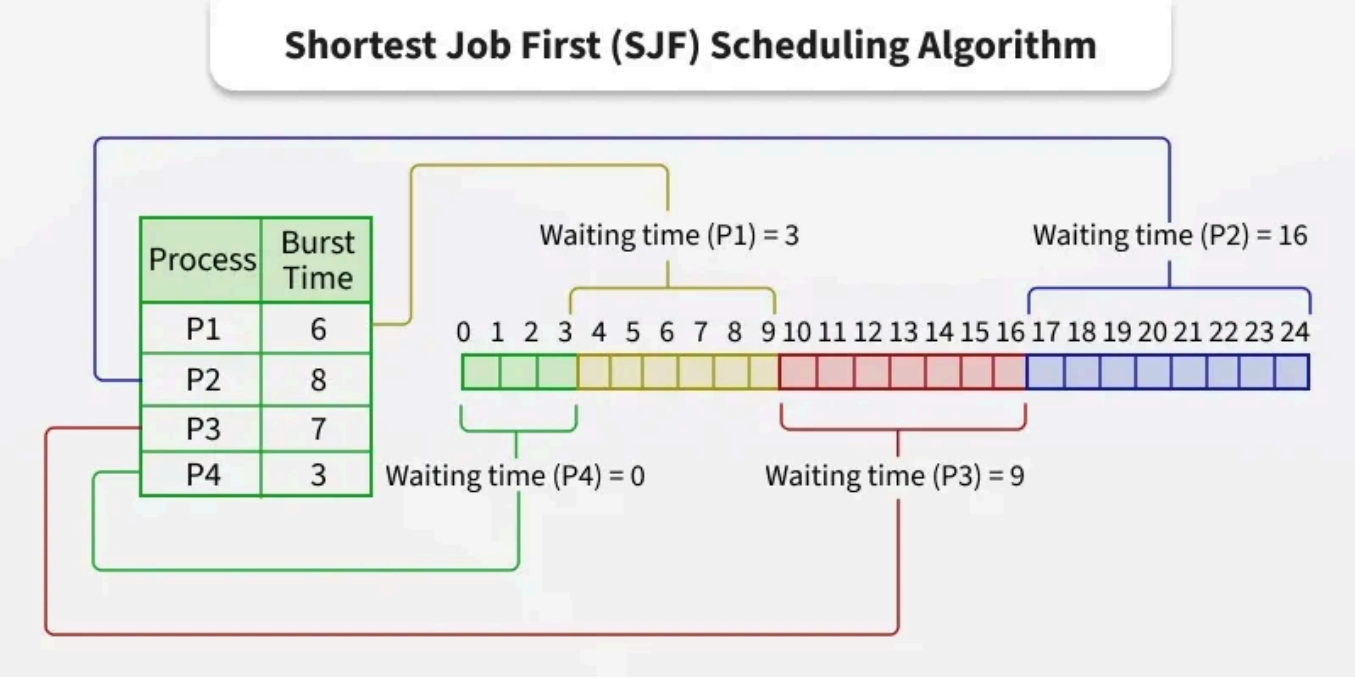
6. Calculate Average Waiting Times

- Average waiting time = Total waiting time / no of processes
- Ex: 10/3 = 3.34

Note - Checking Answers

- The waiting time for each process + Arrival time = The point at which the process began execution on the timeline
- Ex: Process 2 has the wait time of 4. It's arrival time is 1. On the Gantt chart, process 2 began execution at 5. This proves that the calculations were correct.

SJF(Shortest Job First)



- Shortest Job First (SJF) or Shortest Job Next (SJN) is a scheduling process that selects the waiting process with the smallest execution time to execute next.
- This scheduling method may or may not be preemptive.
- Significantly reduces the average waiting time for other processes waiting to be executed.

Characteristics of SJF Scheduling

- SJF allocates CPU to the process with shortest execution time.
- In cases where two or more processes have the same burst time, arbitration is done among these processes on first come first serve basis.
- There are both preemptive and non-premptive versions.
- It minimizes the average waiting time of the processes.
- It may cause starvation of long processes if short processes continue to come in the system.

SJF non-preemptive scheduling

- In the non-preemptive version, once a process is assigned to the CPU, it runs into completion.
- Here, the short term scheduler is invoked when a process completes its execution or when a new process(es) arrives in an empty ready queue.

SJF preemptive scheduling

- This is the preemptive version of SJF scheduling and is also referred as Shortest Remaining Time First (SRTF) scheduling algorithm.
- Here, if a short process enters the ready queue while a longer process is executing, process switch occurs by which the executing process is swapped out to the ready queue while the newly arrived shorter process starts to execute.

- Thus the short term scheduler is invoked either when a new process arrives in the system or an existing process completes its execution.

Examples of Non-Preemptive SJF Algorithm

Example 1

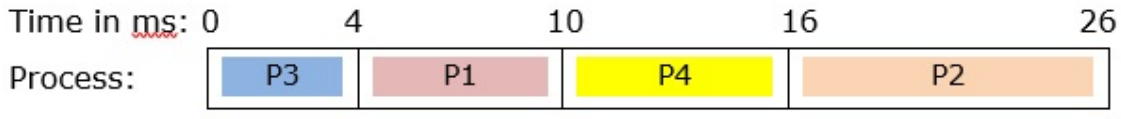
Suppose that we have a set of four processes that have arrived at the same time in the order P1, P2, P3 and P4. The burst time in milliseconds of each process is given by the following table –

Process	CPU Burst Time in ms
P1	6
P2	10
P3	4
P4	6

Let us draw the GANTT chart and find the average turnaround time and average waiting time using non-preemptive SJF algorithm.

GANTT Chart for the set of processes using SJF

Process P3 has the shortest burst time and so it executes first. Then we find that P1 and P4 have equal burst time of 6ms. Since P1 arrived before, CPU is allocated to P1 and then to P4. Finally P2 executes. Thus the order of execution is P3, P1, P4, P2 and is given by the following GANTT chart



Let us compute the average turnaround time and average waiting time from the above chart.

> Average Turnaround Time = Sum of Turnaround Time of each Process / Number of Processes

= (TATP1 + TATP2 + TATP3 + TATP4) / 4

= (10 + 26 + 4 + 16) / 4 = 14 ms

> Average Waiting Time = Sum of Waiting Time of Each Process / Number of processes

= (WTP1 + WTP2 + WTP3 + WTP4) / 4

= (4 + 16 + 0 + 10) / 4 = 7.5 ms

Example 2

In the previous example, we had assumed that all the processes had arrived at the same time, a situation which is practically impossible. Here, we consider circumstance when the processes arrive at different times. Suppose we have set of four processes whose arrival times and CPU burst times are as follows –

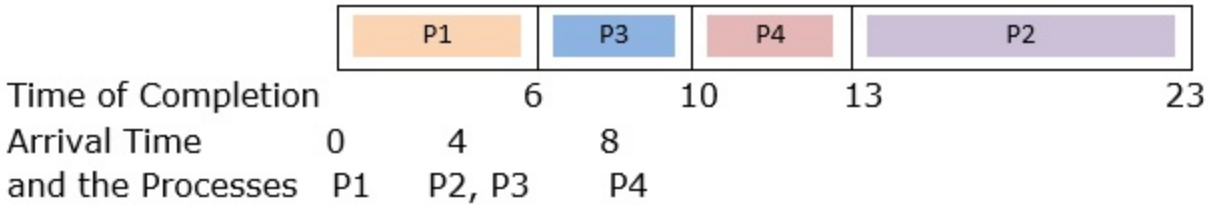
Process	Arrival Time	CPU Burst Time
P1	0	6
P2	4	10
P3	4	4
P4	8	3

Let us draw the GANTT chart and find the average turnaround time and average waiting time using non-preemptive SJF algorithm.

GANTT Chart

While drawing the GANTT chart, we will consider which processes have arrived in the system when the scheduler is invoked. At time 0ms, only P1 is there and so it is assigned to CPU. P1 completes execution at 6ms and at that time P2 and P3 have arrived, but not P4. P3 is assigned to CPU since it has the shortest burst time among current processes. P3 completes execution at time 10ms. By that time P4 has arrived and so SJF algorithm is run on the processes

P2 and P4. Hence, we find that the order of execution is P1, P3, P4, P2 as shown in the following GANTT chart



Let us calculate the turnaround time of each process and hence the average.

> Turnaround Time of a process = Completion Time - Arrival Time

TATP1 = CTP1 - ATP1 = 6 - 0 = 6 ms

TATP2 = CTP2 - ATP2 = 23 - 4 = 19 ms

TATP3 = CTP3 - ATP3 = 10 - 4 = 6 ms

TATP4 = CTP4 - ATP4 = 13 - 8 = 5 ms

> Average Turnaround Time = Sum of Turnaround Time of each Process/ Number of Processes

= (6 + 19+ 6 + 5) / 4 = 9 ms

- The waiting time is given by the time that each process waits in the ready queue. For a non-preemptive scheduling algorithm, waiting time of each process can be simply calculated as –

> Waiting Time of any process = Start Time - CPU Arrival Time

WTP1 = 0 - 0 = 0 ms

WTP2 = 13 - 4 = 9 ms

WTP3 = 6 - 4 = 2 ms

WTP4 = 10 - 8 = 2 ms

> Average Waiting Time = Sum of Waiting Time of Each Process/ Number of processes

= (WTP1 + WTP2 + WTP3 + WTP4) / 4

= (0 + 9 + 2 + 2) / 4 = 3.25 ms

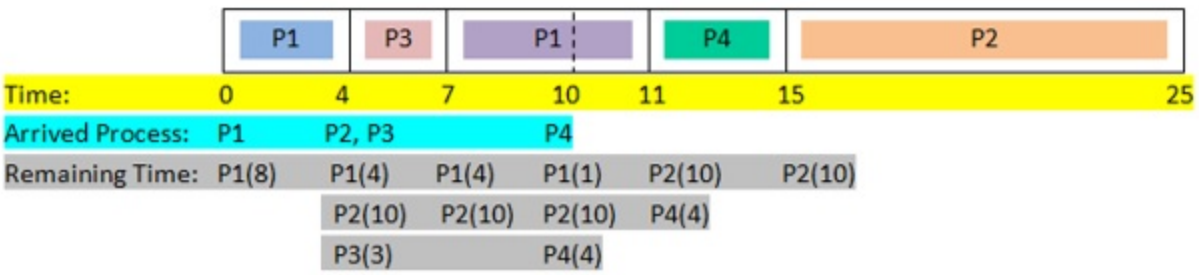
Example of Preemptive SJF (SRTF) algorithm

Let us now perform preemptive SJF (SRTN) scheduling on the following processes, draw GANTT chart and find the average turnaround time and average waiting time.

Process	Arrival Time	CPU Burst Time
P1	0	8
P2	4	10
P3	4	3
P4	10	4

GANTT Chart

Since this is a preemptive scheduling algorithm, the scheduler is invoked when a process arrives and when it completes execution. The scheduler computes the remaining time of completion for each of the processes in the system and selects the process having shortest remaining time left for execution.



Initially, only P1 arrives and so it is assigned to CPU. At time 4ms, P2 and P3 arrive. The scheduler computes the remaining time of the processes P1, P2

and P3 as 4ms, 10ms and 3ms. Since, P3 has shortest time, P1 is pre-empted by P3. P3 completes execution at 7ms and the scheduler is invoked. Among the processes in the system, P1 has shortest time and so it executes. At time 10ms, P4 arrives and the scheduler again computes the remaining times left for each process. Since the remaining time of P1 is least, no process switch occurs and P1 continues to execute. In the similar fashion, the rest of the processes complete execution.

From the GANTT chart, we compute the average turnaround time and the average waiting time.

> Average Turnaround Time = Sum of Turnaround Time of each Process/ Number of Processes

= (TATP1 + TATP2 + TATP3 + TATP4) / 4

= ((11 - 0) + (25 - 4) + (7 - 4) + (15 - 10)) / 4 = 10 ms

> Average Waiting Time = Sum of Waiting Time of Each Process/ Number of processes

= (WTP1 + WTP2 + WTP3 + WTP4) / 4

= (3 + 11 + 0 + 1) / 4 = 3.75 ms

Advantages of SJF Algorithm

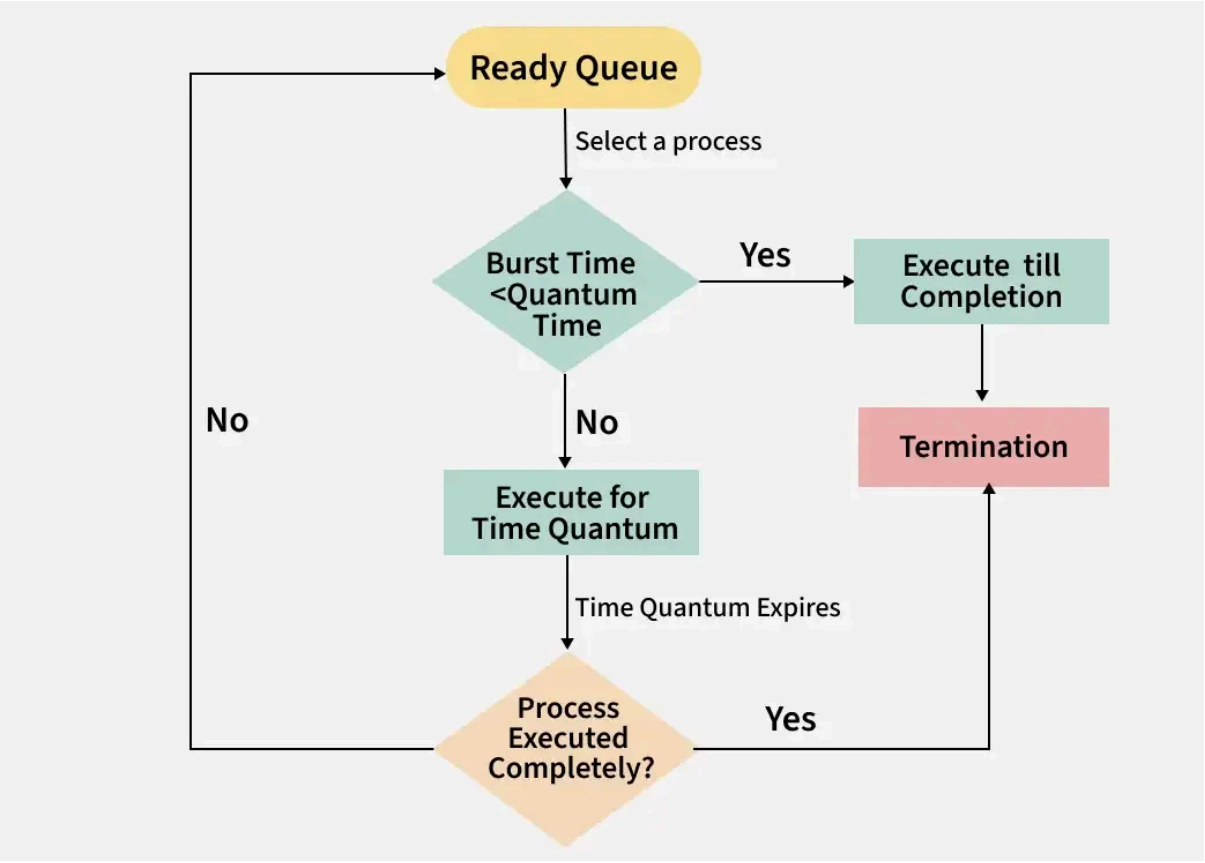
- In both preemptive and non-preemptive methods, the average waiting time is reduced substantially in SJF when compared to FCFS scheduling.
- SJF optimizes turnaround time to a considerable degree.
- If execution time of each process is estimated precisely, it promises maximum throughput.
- Shortest Job first has the advantage of having a *minimum average waiting time among all operating system scheduling algorithms.*

Disadvantages of SJF Algorithm

- In situations where there is an incoming stream of processes with short burst times, longer processes in the system may be waiting in the ready queue indefinitely leading to starvation.
- In preemptive SJF, i.e. SRTF, if all processes arrive at different times and at frequent intervals, the scheduler may be always working and consequently the processor may be more engaged in process switching than actual execution of the processes.
- Correct estimation of the burst time a process is a complicated process. Since the effectiveness of the algorithm is entirely based upon the burst times, an erroneous calculation may cause inefficient scheduling.

Round Robin Scheduling

- It is called "round robin" because the system rotates through all the processes, allocating each of them a fixed time slice or "quantum", regardless of their priority.
- The primary goal of this scheduling method is to ensure that all processes are given an equal opportunity to execute, promoting fairness among tasks.
    - **Process Arrival:** Processes enter the system and are placed in a queue.
    - **Time Allocation:** Each process is given a certain amount of CPU time, called a quantum.
    - **Execution:** The process uses the CPU for the allocated time.
    - **Rotation:** If the process completes within the time, it leaves the system. If not, it goes back to the end of the queue.
    - **Repeat:** The CPU continues to cycle through the queue until all processes are completed.



Scenario 1: Processes with Same Arrival Time

Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2 ms**

Process	Burst Time	Arrival Time
P1	4 ms	0 ms
P2	5 ms	0 ms
P3	3 ms	0 ms

Step-by-Step Execution:

1. Time 0-2 (P1): P1 runs for 2 ms (total time left: 2 ms).
2. Time 2-4 (P2): P2 runs for 2 ms (total time left: 3 ms).
3. Time 4-6 (P3): P3 runs for 2 ms (total time left: 1 ms).
4. Time 6-8 (P1): P1 finishes its last 2 ms.
5. Time 8-10 (P2): P2 runs for another 2 ms (total time left: 1 ms).

6. Time 10-11 (P3): P3 finishes its last 1 ms.
7. Time 11-12 (P2): P2 finishes its last 1 ms.
   Now, lets calculate average waiting time and turn around time:

> - **Turnaround Time** = Completion Time - Arrival Time
> - **Waiting Time** = Turnaround Time - Burst Time

Processes	AT	BT	CT	TAT	WT
P1	0	4	8	8-0 = 8	8-4 = 4
P2	0	5	12	12-0 = 12	12-5 = 7
P3	0	3	11	11-0 = 11	11-3 = 8

- **Average Turn around time** = (8 + 12 + 11)/3 = 31/3 = 10.33 ms
- **Average waiting time** = (4 + 7 + 8)/3 = 19/3 = 6.33 ms

Scenario 2: Processes with Different Arrival Times

Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2**

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	0 ms
P2	2 ms	4 ms
P3	4 ms	5 ms

Step-by-Step Execution:

1. Time 0-2 (P1 Executes):

- P1 starts execution as it arrives at 0 ms.
- Runs for 2 ms; remaining burst time = 5 - 2 = 3 ms.
- **Ready Queue:** [P1].

2. Time 2-4 (P1 Executes Again):

- P1 continues execution since no other process has arrived yet.
- Runs for 2 ms; remaining burst time = 3 - 2 = 1 ms.
- P2 arrive at 4 ms.
- **Ready Queue:** [P2, P1].

3. Time 4-6 (P2 Executes):

- P2 starts execution as it arrives at 4 ms.
- Runs for 2 ms; remaining burst time = 2 - 2 = 0 ms.
- P3 arrive at 5ms
- **Ready Queue:** [P1, P3].

4. Time 6-7 (P1 Executes):

- P1 starts execution.
- Runs for 1 ms; remaining burst time = 1 - 1 = 0 ms.
- **Ready Queue:** [P3].

5. Time 7-9 (P3 Executes):

- P3 starts execution.
- Remaining burst time = 4 - 2 = 2 ms.
- **Ready Queue:** [P3].

6. Time 9-11 (P3 Executes Again):

- P3 resumes execution and runs for 2 ms and complete its execution
- Remaining burst time = 2 - 2 = 0 ms.
- **Ready Queue:** [].